

Out-of-Order 2-Way Superscalar IITB-RISC Processor

Project Report

Team ID: 39

Team Members

All members contributed equally, listed in alphabetical order

Advay Bapat (23B1225)

Apoorv Goyal (23B1266)

Shreya Nigam (23B1258)

Suketu Patni (23B1299)

Department of Electrical Engineering

IIT Bombay

May 12, 2026

Contents

1	Introduction and Architectural	
	Overview	3
1.1	Design Objectives	3
1.2	Microarchitectural Summary	3
2	Detailed Design Modules	4
2.1	Front-End Dispatch and Register Renaming	4
2.2	Distributed Reservation Stations and Schedulers	4
2.2.1	ALU Reservation Station	4
2.2.2	Branch Reservation Station	5
2.2.3	Load/Store Reservation Station	5
2.3	Execution Pipelines and the Common Data Bus	5
2.4	Memory Subsystem: Store Buffer and Store-to-Load Forwarding	5
2.5	In-Order Commit Through the Reorder Buffer	6
2.6	LM/SM Block	6
3	Simulation Results and Verification	7
3.1	Test 1: LLI	8
3.2	Test 2: ADI, ADA, NDU	9
3.3	Test 3: ADC	10
3.4	Test 4: ADZ	11
3.5	Test 5: AWC	12
3.6	Test 6: ACA	13
3.7	Test 7: ACC	14
3.8	Test 8: ACZ	15
3.9	Test 9: ACW	16
3.10	Test 10: NDC	17
3.11	Test 11: NDZ	18
3.12	Test 12: NCU	19
3.13	Test 13: NCC	20
3.14	Test 14: NCZ	21
3.15	Test 15: SW and LW	22
3.16	Test 16: BEQ	23
3.17	Test 17: BLE	24
3.18	Test 18: BLT	25
3.19	Test 19: LM	26
3.20	Test 20: JLR	27
3.21	Test 21: JRI	28
3.22	Test 22: JAL	29
3.23	Test 23: R0	30

Contents	2
4 Conclusion	31
5 Assembler	32
6 Work Distribution	37

1 Introduction and Architectural Overview

This project describes the design and VHDL implementation of an out-of-order, 2-way superscalar backend for the IITB-RISC instruction set architecture. The implementation was developed to support dual instruction dispatch, dynamic scheduling, speculative operand wakeup, and precise in-order retirement while preserving architectural correctness under data and control dependence constraints.

The processor was organized around an execution model with distributed reservation stations, decoupled schedulers, a common data bus, and a reorder buffer. Operand availability was resolved dynamically through tag-based wakeup, while architectural state was committed strictly in program order to guarantee precise state recovery. The backend was further extended with a dedicated store buffer and store-to-load forwarding mechanism to reduce memory latency and preserve memory ordering semantics.

A 2-way fetch and dispatch target was adopted to exploit instruction-level parallelism at a modest implementation complexity. The design was therefore partitioned into independent execution clusters: dual ALU pipelines, a branch pipeline, a load/store unit, and a memory-retirement subsystem. Each cluster was connected through a unified communication fabric, allowing results, flags, and control tokens to be broadcast to dependent instructions without centralized scoreboarding.

1.1. Design Objectives

The principal objectives of the project were as follows:

- to support two instructions per cycle at dispatch,
- to resolve register and flag dependencies using rename structures,
- to issue instructions out of order while preserving in-order retirement,
- to maintain precise exception and recovery semantics through a reorder buffer,
- to support store buffering and store-to-load forwarding for memory efficiency, and
- to verify the implementation using cycle-accurate VHDL testbenches.

1.2. Microarchitectural Summary

The final backend may be summarized by the following structural choices:

Feature	Implementation Choice
Dispatch width	2-way
Scheduling model	Dynamic scheduling
Reservation stations	Distributed RS for ALU, branch, and LSU
Execution latency	1-cycle ALU, 2-cycle branch and LSU
Broadcast	4-lane common data bus
Commit model	Strict in-order retirement through ROB
Memory subsystem	Store buffer with store-to-load forwarding
Flag handling	Split zero/carry rename domain

2 Detailed Design Modules

2.1. Front-End Dispatch and Register Renaming

The dispatcher was designed to accept up to two instructions per cycle and to route them to the appropriate backend queue. A register alias table was used to map architectural registers to rename registers, thereby eliminating false dependencies and enabling out-of-order issue.

A split-flag architecture was adopted because IITB-RISC instructions do not uniformly update the same status information. For example, some operations update both carry and zero flags, while others update only one of them or do not modify flags at all. Accordingly, the rename domain was partitioned into a general-purpose data rename file, a zero-flag rename file, and a carry-flag rename file. This prevented independent flag-producing instructions from corrupting each other through shared state.

Sequential PC updates were handled outside the reservation stations, whereas explicit writes to R0 were routed through the branch execution path. This eliminated unbreakable read-after-write hazards associated with control-flow instructions and allowed PC-dependent behavior to be preserved without stalling the frontend unnecessarily.

2.2. Distributed Reservation Stations and Schedulers

A reservation-station centric architecture was employed to decouple dispatch from execution. Each functional domain maintained its own queue and scheduler.

2.2.1. ALU Reservation Station

The ALU reservation station was implemented as an 8-entry out-of-order waiting structure with 2-way enqueue capability. Its entries held operand tags, operand values, destination tags, instruction control bits, and readiness information. The scheduler selected the first two ready entry when the operands had been fully resolved and the downstream ALU pipeline had asserted availability.

2.2.2. Branch Reservation Station

The branch reservation station was implemented as a small FIFO queue, sized to keep only a few unresolved branches in flight. The design was intentionally conservative, so that no more than three or four unresolved branches would be permitted simultaneously. Once the number of remaining free slots dropped below the 2-way fetch requirement, dispatch was stalled.

Branches were handled using direct operand comparison rather than by relying on global ALU flags. This decision was consistent with the ISA structure, in which branch outcomes are derived from explicit register comparisons.

2.2.3. Load/Store Reservation Station

The LSU reservation station held memory operations awaiting operand resolution or address generation. The design used explicit busy bits, 2-way enqueue, broad wakeup through the common data bus, and same-cycle forwarding for newly dispatched entries when operand tags matched an already broadcast result. In fact, this feature is in all three reservation stations. This allowed load/store operations to participate in the same dynamic-scheduling fabric as arithmetic and branch instructions.

2.3. Execution Pipelines and the Common Data Bus

The backend used a four-lane common data bus to propagate results from the execution clusters to all waiting reservation stations. Each lane carried the result value together with the associated tag information required to wake dependent instructions.

The bus was structured to broadcast result data and the corresponding update tokens for the data path and status flags in the same cycle. This allowed a single producer to wake multiple dependent consumers across multiple functional domains without centralized arbitration.

The execution subsystem was partitioned into three principal pipelines:

- two parallel ALU pipelines with one-cycle latency,
- one branch pipeline with two-cycle latency, and
- one LSU pipeline with two-cycle latency.

The differing latencies were explicitly modeled in the verification environment. In particular, the branch and LSU paths were treated as longer-latency resources, and the testbench was synchronized to their delayed completion behavior to ensure cycle-accurate verification.

2.4. Memory Subsystem: Store Buffer and Store-to-Load Forwarding

A dedicated store buffer was introduced to preserve in-order store retirement while permitting address generation and data availability to proceed independently. Stores were dispatched into both the LSU path and the store buffer. The LSU computed the store address, while the store buffer retained the associated data and address until the ROB authorized retirement.

The buffer was designed to support store-to-load forwarding. If a load address matched a pending store address, the store buffer forwarded the data directly to the load path, thereby bypassing the data cache and reducing memory latency. If the address matched but the corresponding store data had not yet arrived, the load was stalled until the store became resolvable.

To prevent contention on the memory interface, arbitration was enforced between the store-retirement path and the LSU load path. The store-retire controller was given priority when a commit required cache interaction, while the LSU used the address bus during ordinary load execution.

2.5. In-Order Commit Through the Reorder Buffer

Although instructions were allowed to issue out of order, retirement was performed strictly in program order through the reorder buffer. This ensured precise architectural state and deterministic recovery after branch misprediction or other pipeline disruption.

The ROB tracked each instruction from dispatch until retirement. Once the head entry became complete, its architectural side effects were committed. Stores were written to the memory system only when they reached the head of the ROB, thereby preventing speculative memory writes from becoming architecturally visible too early.

A conditional architectural register-file write structure was used to preserve flag semantics. In particular, instructions such as ADC and ADZ were handled using a control-aware writeback selection mechanism that determined whether the integer datapath, zero flag, carry flag, or program-counter state should be updated at commit time.

2.6. LM/SM Block

The `Multi_Inst_Block` is a hardware microcode sequencer designed to unroll LM/SM instructions into a sequence of standard LW and SW operations. If instruction 1 contains an LM/SM, instruction 2 is stalled and routed back to the fetch stage, whereas if only instruction 2 is an LM/SM, instruction 1 executes normally and instruction 2 is fed back to be processed in the primary slot on the next cycle. When processing an LM/SM from IR, the block extracts up to two target registers per cycle from a 9-bit mask; the first encoder identifies the lowest set bit, assigns its index, and clears it to produce a temporary instruction. Two full adders calculate the necessary memory offsets by adding 2 and 4. The block then constructs up to two standard memory instructions by splicing the original opcode, the base address register `reg_a`, the extracted destination registers, and the calculated immediate offsets, while outputting the next base offset and the total remaining register count. If an odd number of registers is left, the instruction for the last register is duplicated.

3 Simulation Results and Verification

This section presents cycle-accurate simulation results for representative test programs used to validate the processor backend. The simulations were designed to exercise instruction dispatch, dependency resolution, reservation-station wakeup, execution latency, store forwarding, and in-order retirement.

3.1. Test 1: LLI

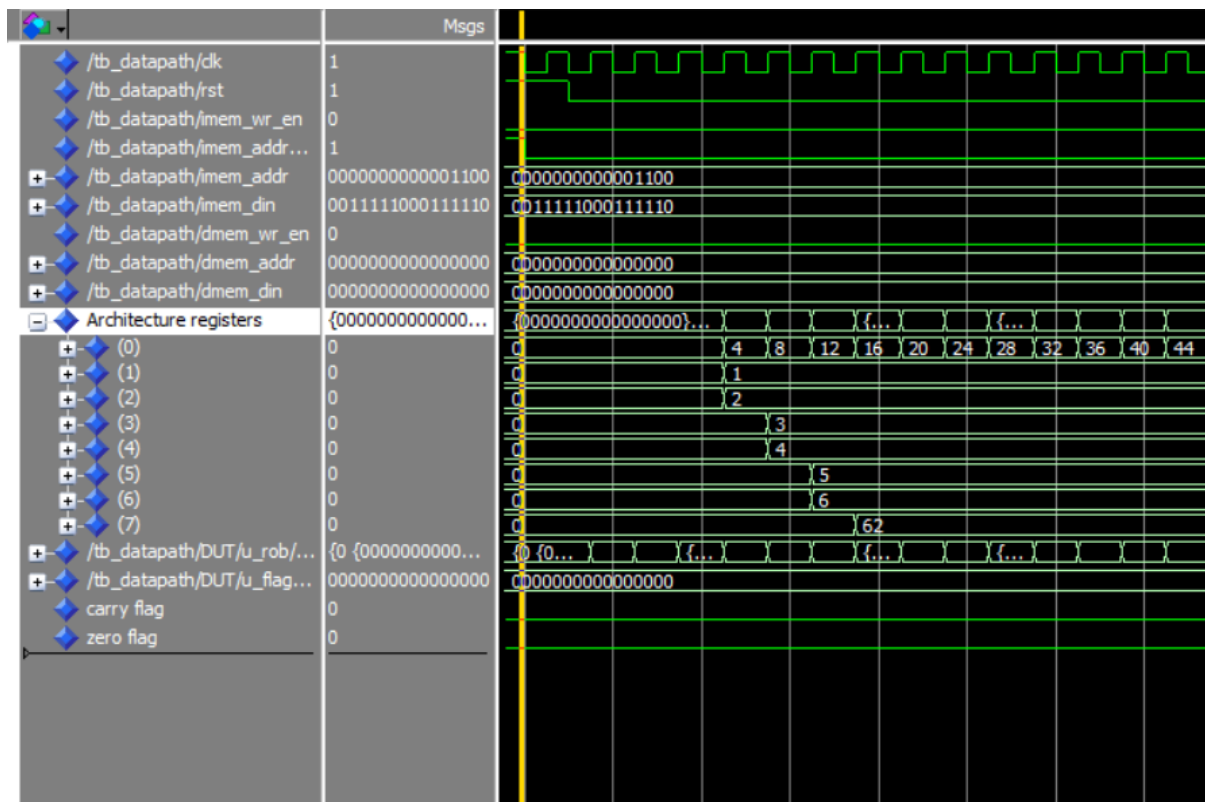


Figure 1: Simulation waveform for the LLI instruction.

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
00110110000000011 // lli r3, 3
0011100000000100 // lli r4, 4
0011101000000101 // lli r5, 5
0011110000000110 // lli r6, 6
00111110001111110 // lli r7, 62

```

3.2. Test 2: ADI, ADA, NDU

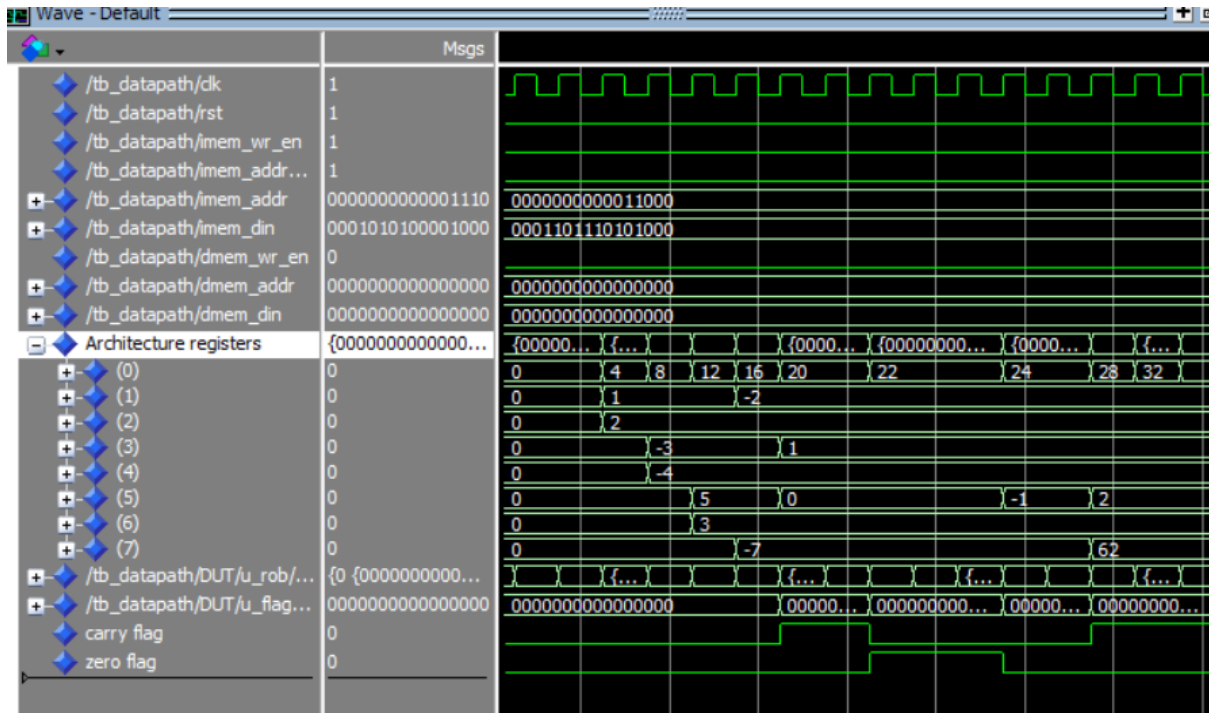


Figure 2: Simulation waveform for ADI, ADA, and NDU instructions.

```

0011001000000001 // lli r1, 1
0011010000000010 // lli r2, 2
0000011011111101 // adi r3, r3, -3
0000100100111100 // adi r4, r4, -4
0011101000000101 // lli r5, 5
0001001010110000 // ada r6, r1, r2
0001011100111000 // ada r7, r3, r4
0001010100001000 // ada r1, r2, r4
0001101100011000 // ada r3, r5, r4
0011101000000000 // lli r5, 0
0001101101101000 // ada r5, r5, r5
0010101101101000 // ndu r5, r5, r5
0001101110101000 // ada r5, r5, r6
0011111000111110 // lli r7, 62

```

3.3. Test 3: ADC

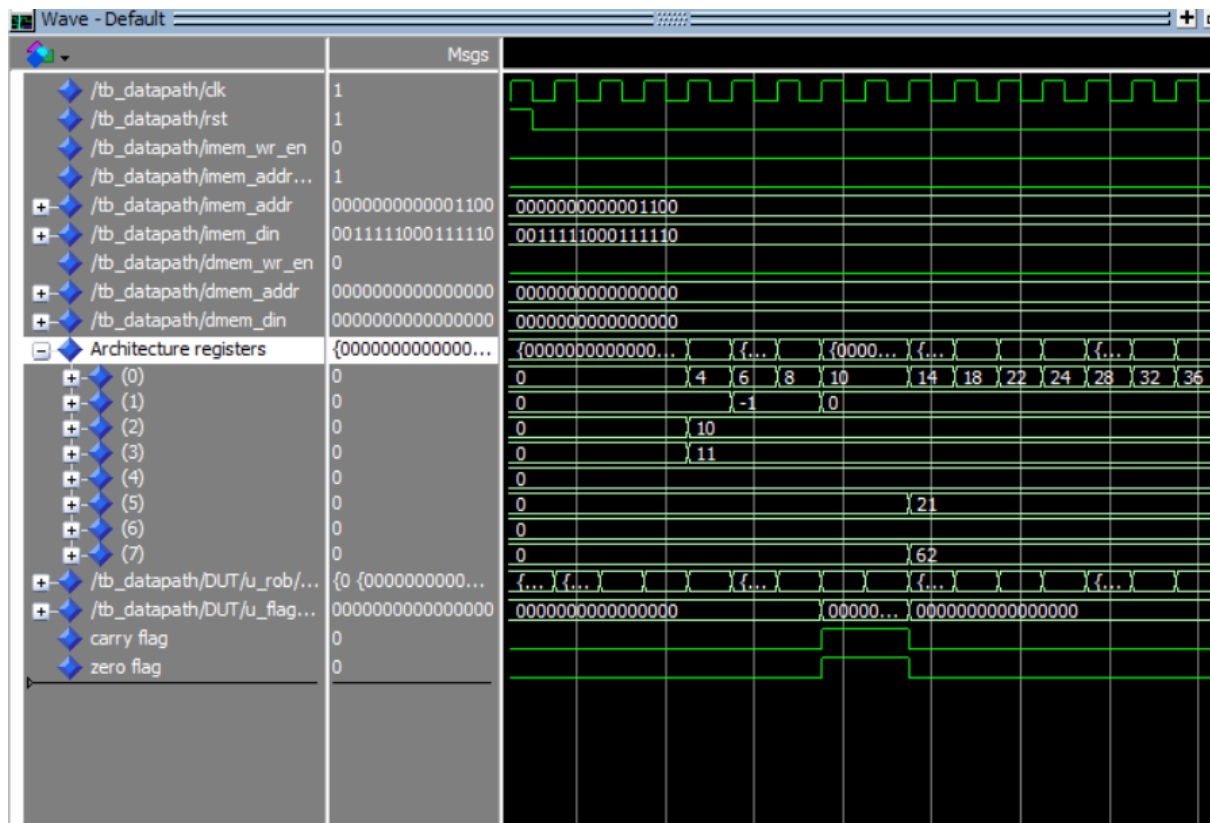


Figure 3: Simulation waveform for the ADC instruction.

```

0011010000001010 // lli r2, 10
0011011000001011 // lli r3, 11
0010001001001000 // ndu r1, r1, r1
0001010011100010 // adc r4, r2, r3
0000001001000001 // adi r1, r1, 1
0001010011101010 // adc r5, r2, r3
0011111000111110 // lli r7, 62

```

3.4. Test 4: ADZ

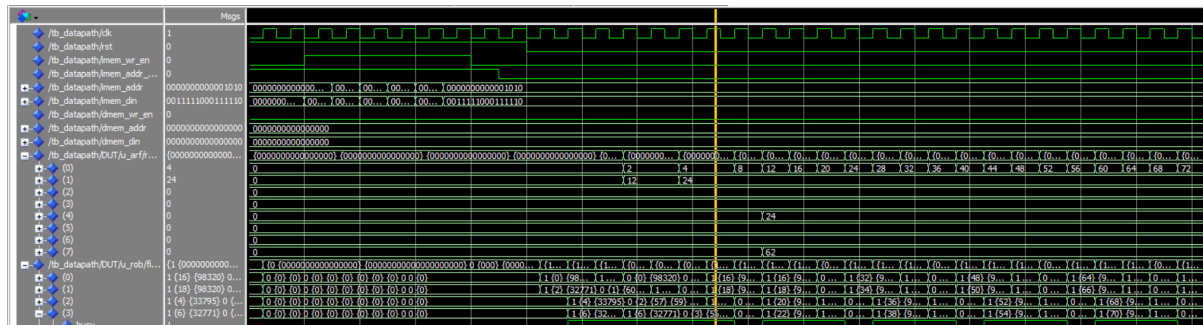


Figure 4: Simulation waveform for the ADZ instruction.

```

0011001000001100 // lli r1, 12
0001001001001000 // ada r1, r1, r1
0001010001010001 // adz r2, r2, r1
0001011011011000 // ada r3, r3, r3
0001100001100001 // adz r4, r4, r1
0011111000111110 // lli r7, 62

```

3.5. Test 5: AWC

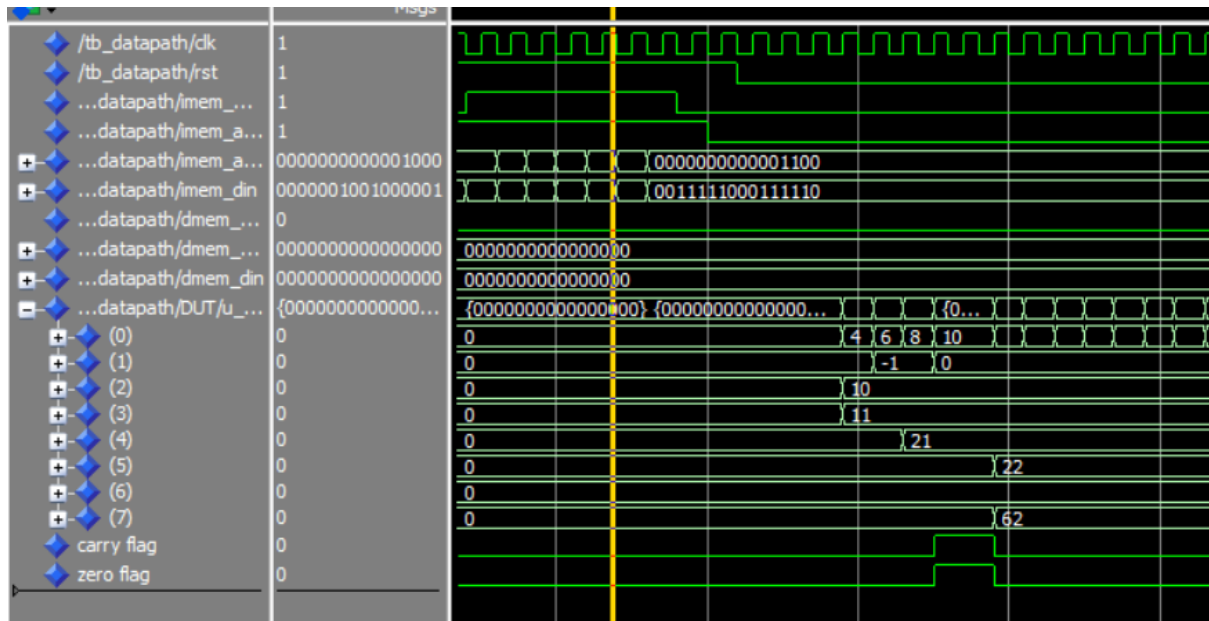


Figure 5: Simulation waveform for the AWC instruction.

```

0011010000001010 // lli r2, 10
0011011000001011 // lli r3, 11
0010001001001000 // ndu r1, r1, r1
0001010011100011 // awc r4, r2, r3
0000001001000001 // adi r1, r1, 1
0001010011101011 // awc r5, r2, r3
0011111000111110 // lli r7, 62

```

3.6. Test 6: ACA

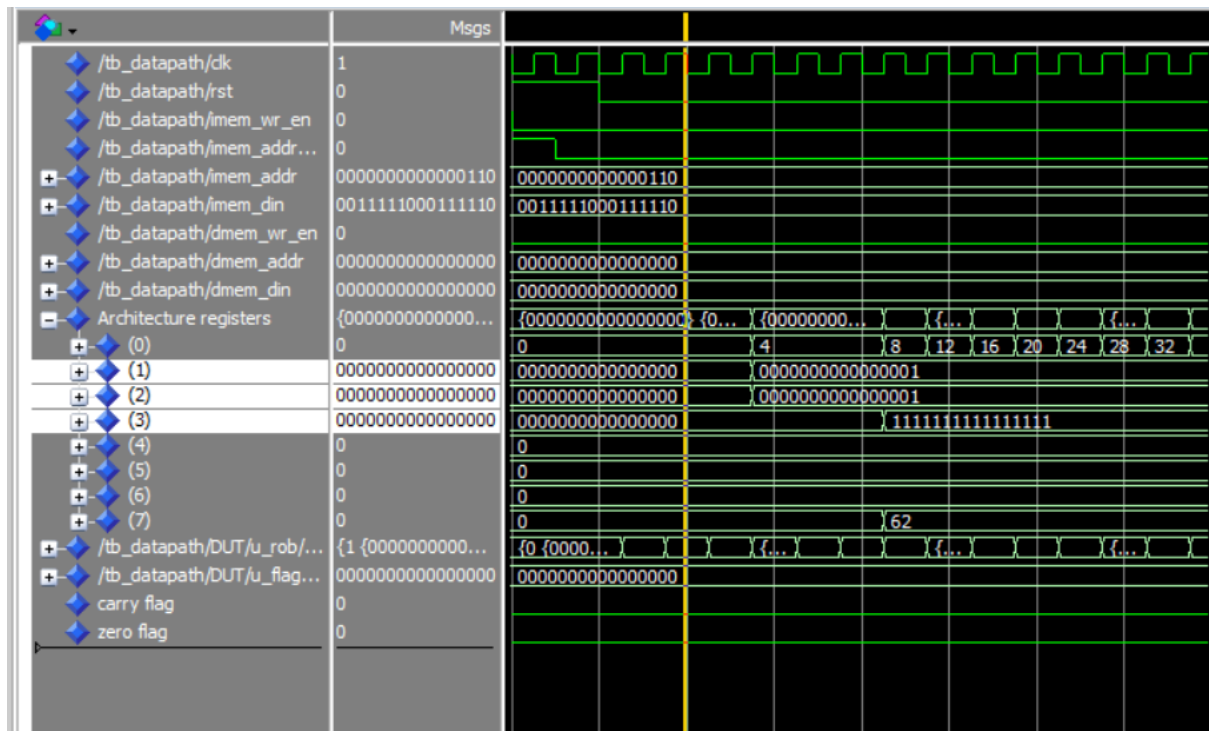


Figure 6: Simulation waveform for the ACA instruction.

```
00110010000000001 // lli r1, 1
00110100000000001 // lli r2, 1
0001001010011100 // aca r3, r1, r2
0011111000111110 // lli r7, 62
```

3.7. Test 7: ACC

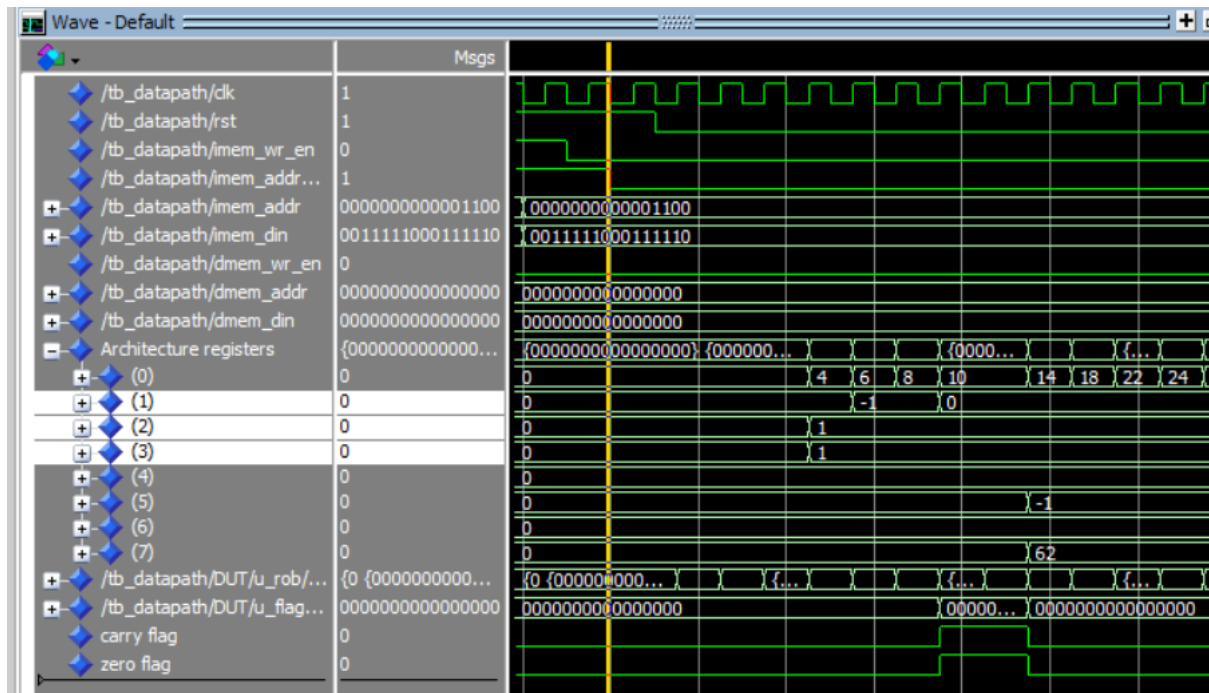


Figure 7: Simulation waveform for the ACC instruction.

```

00110100000000001 // lli r2, 1
00110110000000001 // lli r3, 1
0010001001001000 // ndu r1, r1, r1
0001010011100110 // acc r4, r2, r3
0000001001000001 // adi r1, r1, 1
0001010011101110 // acc r5, r2, r3
0011111000111110 // lli r7, 62

```

3.8. Test 8: ACZ

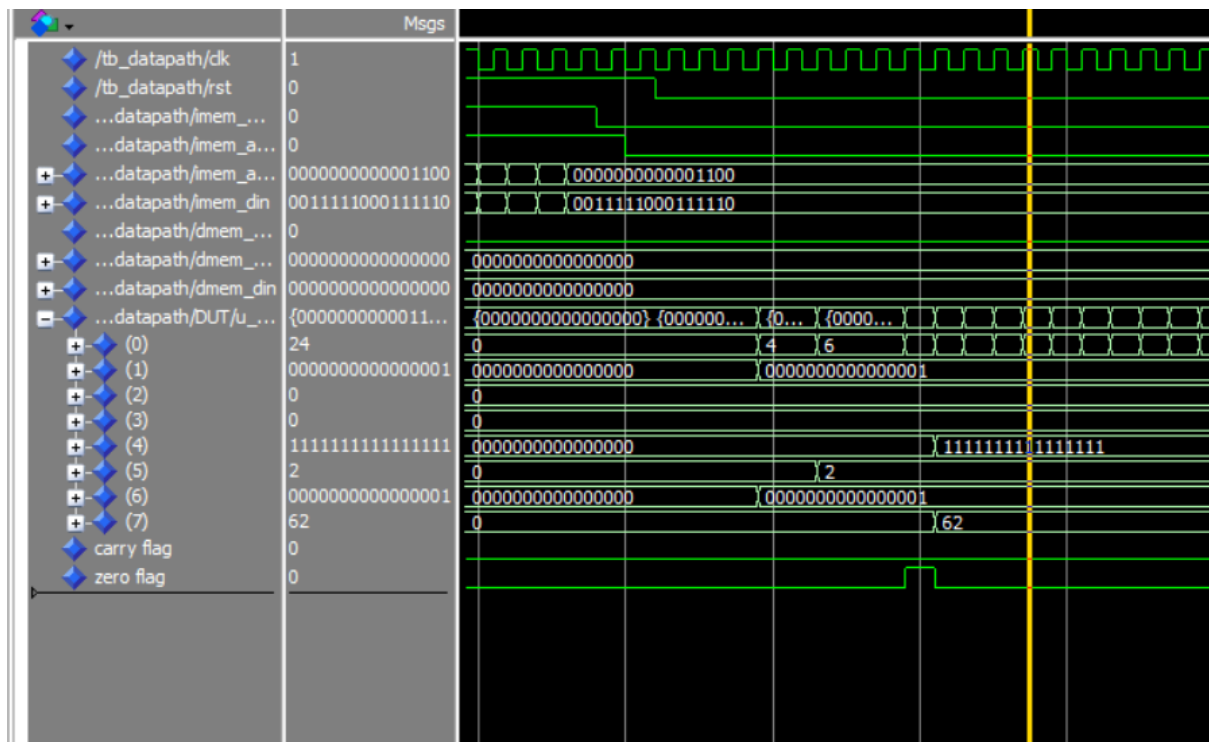


Figure 8: Simulation waveform for the ACZ instruction.

```

00110010000000001 // lli r1, 1
00111110000000001 // lli r6, 1
0001001001101000 // ada r5, r1, r1
0001001110010101 // acz r2, r1, r6
0001011011011000 // ada r3, r3, r3
0001001110100101 // acz r4, r1, r6
0011111000111110 // lli r7, 62

```


3.9. Test 9: ACW

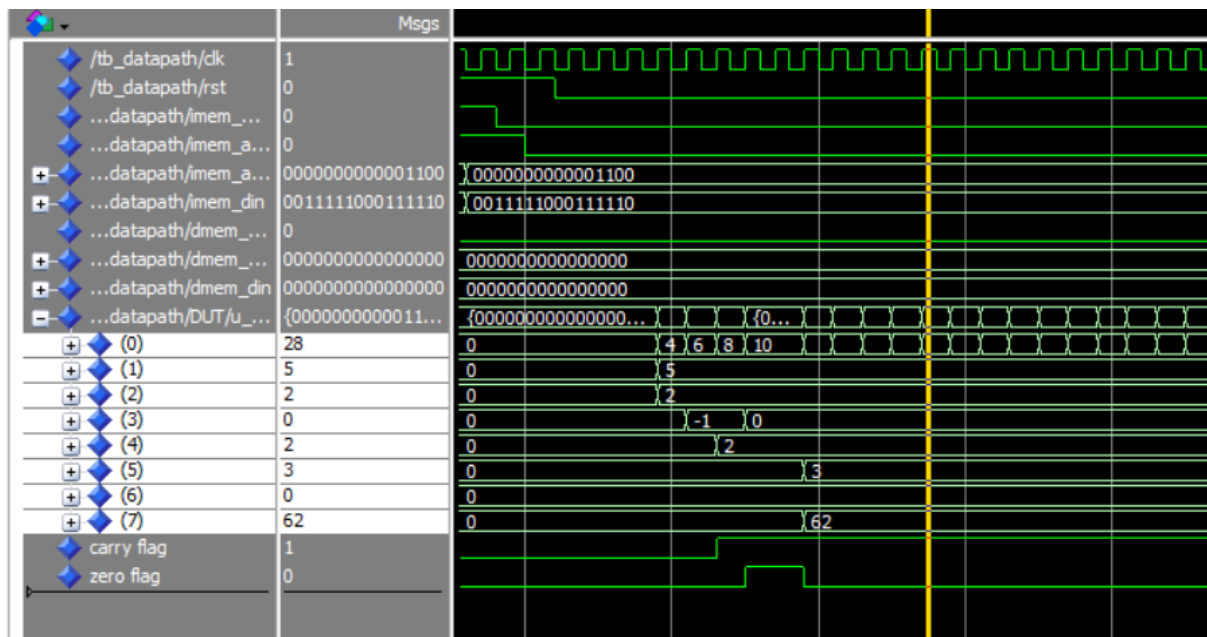


Figure 9: Simulation waveform for the ACW instruction.

```

0011001000000101 // lli r1, 5
0011010000000010 // lli r2, 2
0010011011011000 // ndu r3, r3, r3
0001001010100111 // acw r4, r1, r2
0000011011000001 // adi r3, r3, 1
0001001010101111 // acw r5, r1, r2
0011111000111110 // lli r7, 62

```

3.10. Test 10: NDC

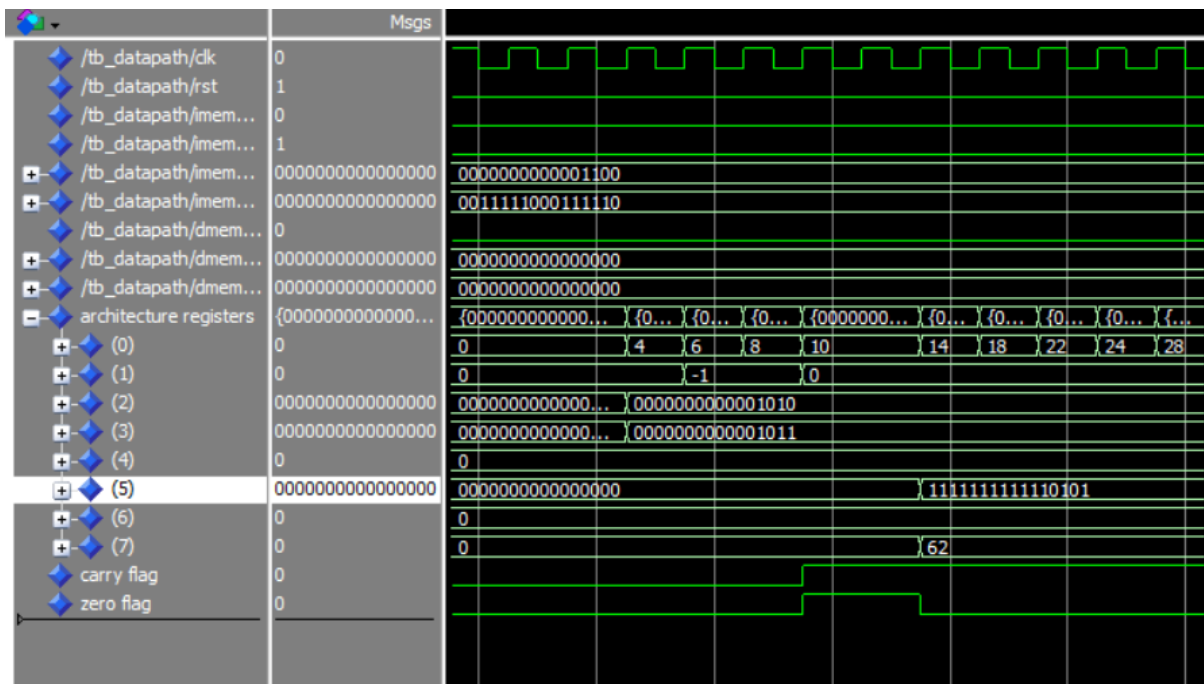


Figure 10: Simulation waveform for the NDC instruction.

```

0011010000001010 // lli r2, 10
0011011000001011 // lli r3, 11
0010001001001000 // ndu r1, r1, r1
0010010011100010 // ndc r4, r2, r3
0000001001000001 // adi r1, r1, 1
0010010011101010 // ndc r5, r2, r3
0011111000111110 // lli r7, 62

```

3.11. Test 11: NDZ

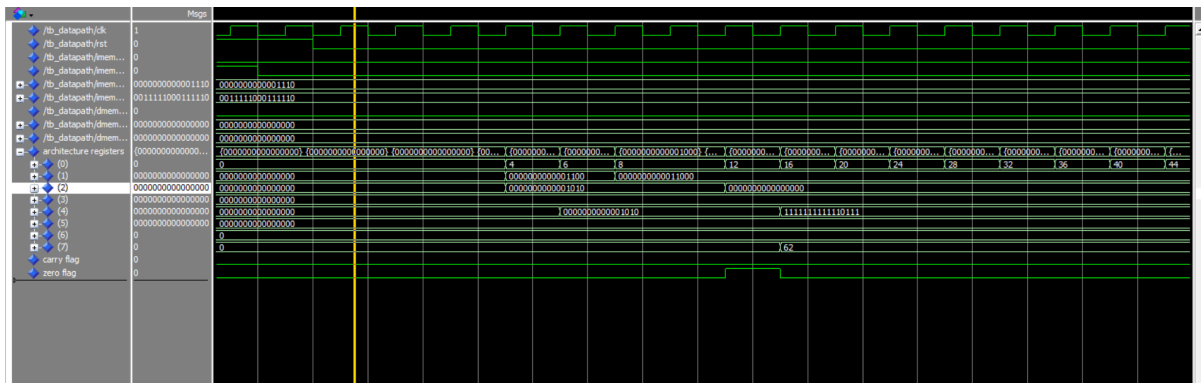


Figure 11: Simulation waveform for the NDZ instruction.

```

00110010000001100 // lli r1, 12
00110100000001010 // lli r2, 10
00111000000001010 // lli r4, 10
0001001001001000 // ada r1, r1, r1
0010010001010001 // ndz r2, r2, r1
0001011011011000 // ada r3, r3, r3
0010100001100001 // ndz r4, r4, r1
00111110001111110 // lli r7, 62

```

3.12. Test 12: NCU

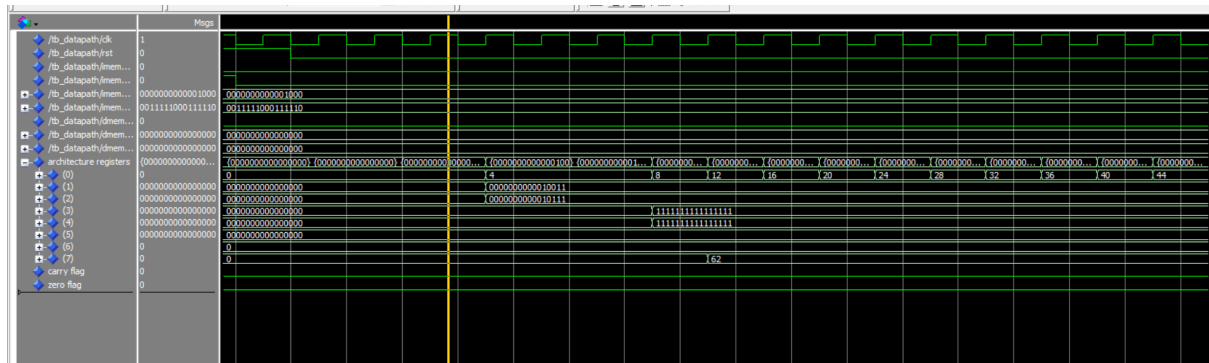


Figure 12: Simulation waveform for the NCU instruction.

```

0011001000010011 // lli r1, 19
0011010000010111 // lli r2, 23
0010001010011100 // ncu r3, r1, r2
0010101010100100 // ncu r4, r5, r2
0011111000111110 // lli r7, 62

```

3.13. Test 13: NCC

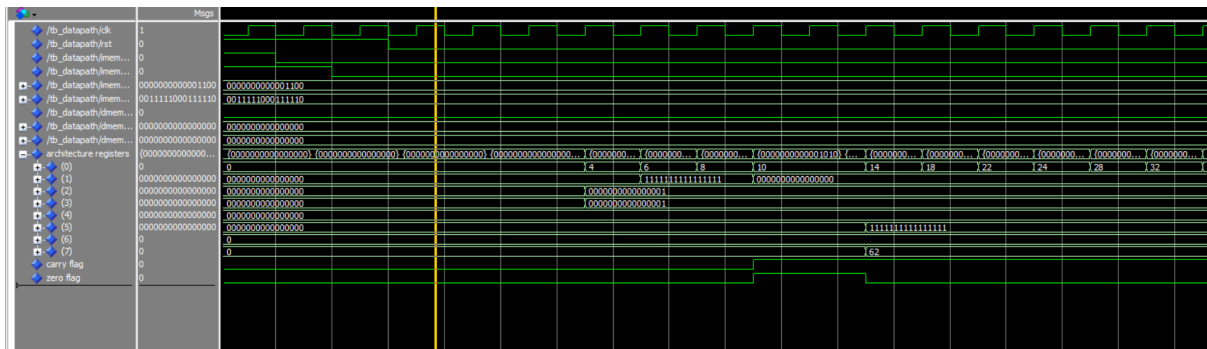


Figure 13: Simulation waveform for the NCC instruction.

```

00110100000000001 // lli r2, 1
00110110000000001 // lli r3, 1
0010001001001000 // ndu r1, r1, r1
0010010011100110 // ncc r4, r2, r3
00000010010000001 // adi r1, r1, 1
0010010011101110 // ncc r5, r2, r3
0011111000111110 // lli r7, 62

```

3.14. Test 14: NCZ

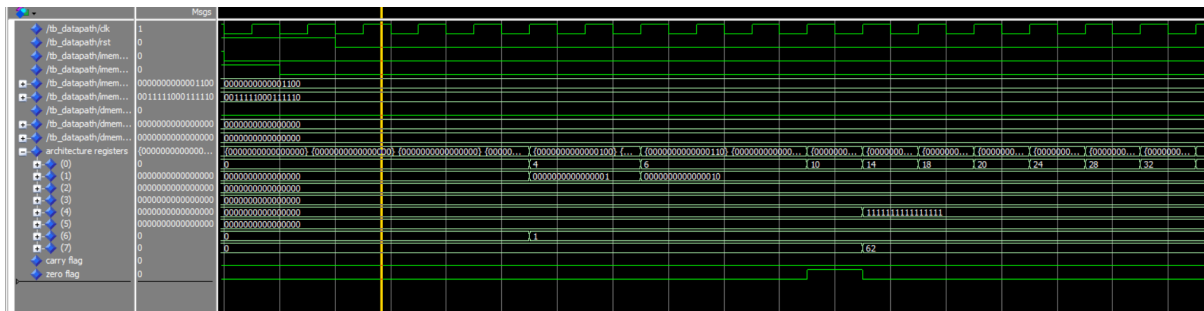


Figure 14: Simulation waveform for the NCZ instruction.

```

00110010000000001 // lli r1, 1
00111100000000001 // lli r6, 1
0001001001001000 // ada r1, r1, r1
0010010010010101 // ncz r2, r2, r2
0001011011011000 // ada r3, r3, r3
0010100110100101 // ncz r4, r4, r6
0011111000111110 // lli r7, 62

```

3.15. Test 15: SW and LW

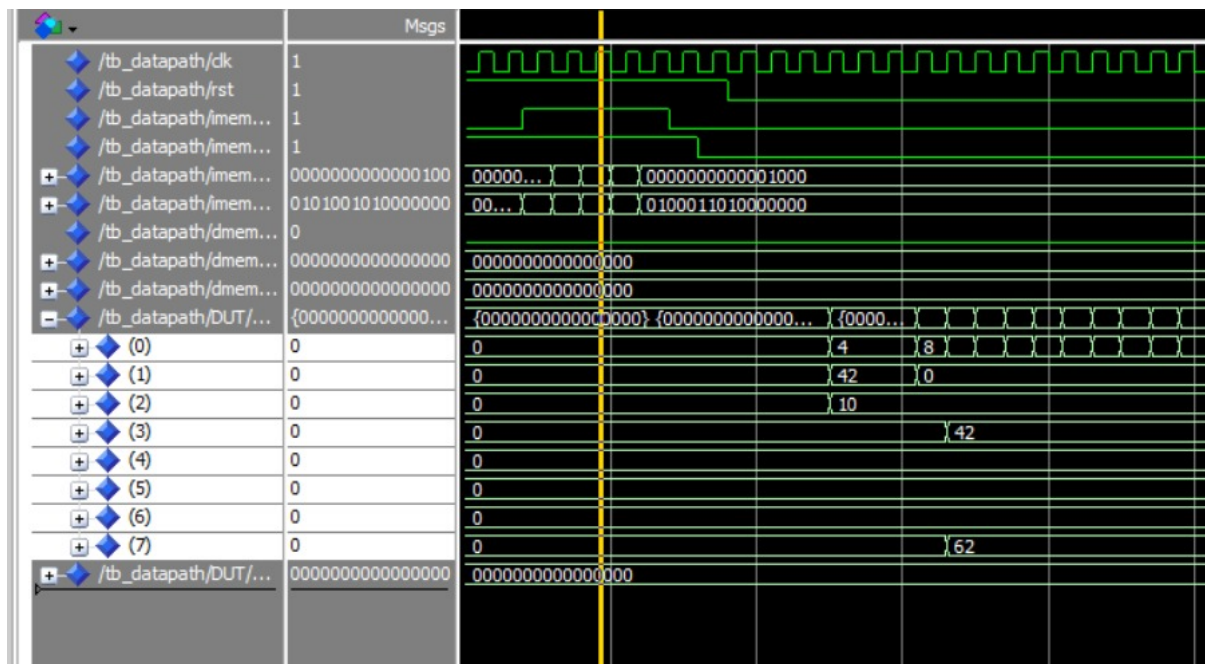


Figure 15: Simulation waveform for the SW and LW instruction.

```

0011001000101010 // lli r1, 42
0011010000001010 // lli r2, 10
0101001010000000 // sw r1, r2, 0
0011001000000000 // lli r1, 0
0100011010000000 // lw r3, r2, 0

```

3.16. Test 16: BEQ

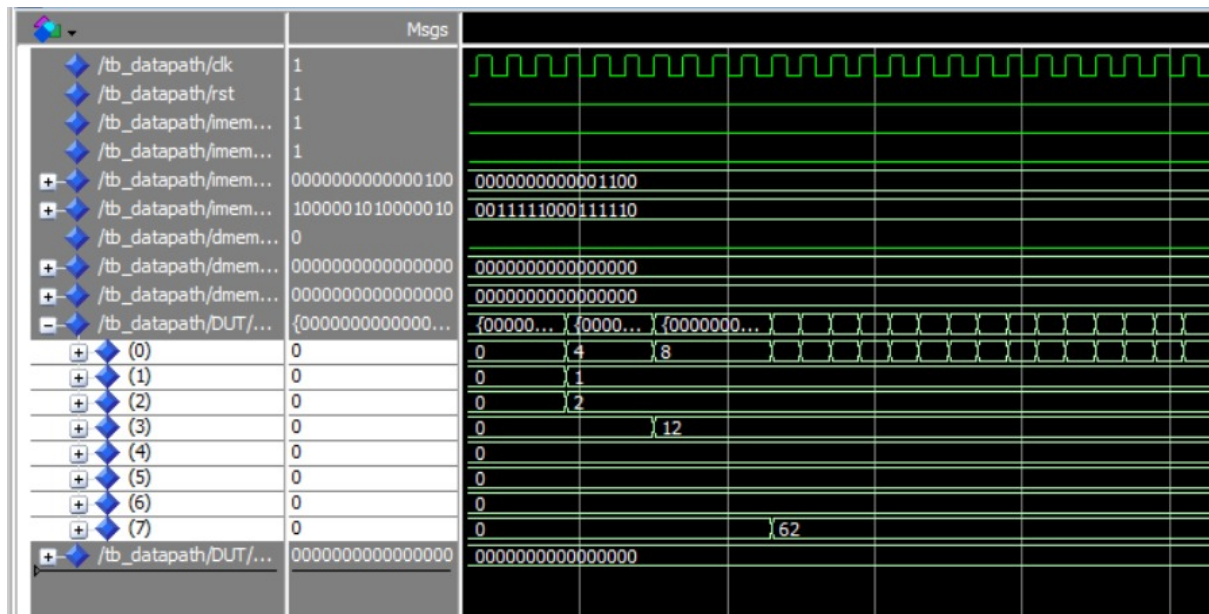


Figure 16: Simulation waveform for the BEQ instruction (Branch if Equal).

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
1000001010000010 // beq r1, r2, 2
0011011000001100 // lli r3, 12
1000011011000010 // beq r3, r3, 2
00111000000000100 // lli r4, 4
00111110001111110 // lli r7, 62

```


3.17. Test 17: BLE

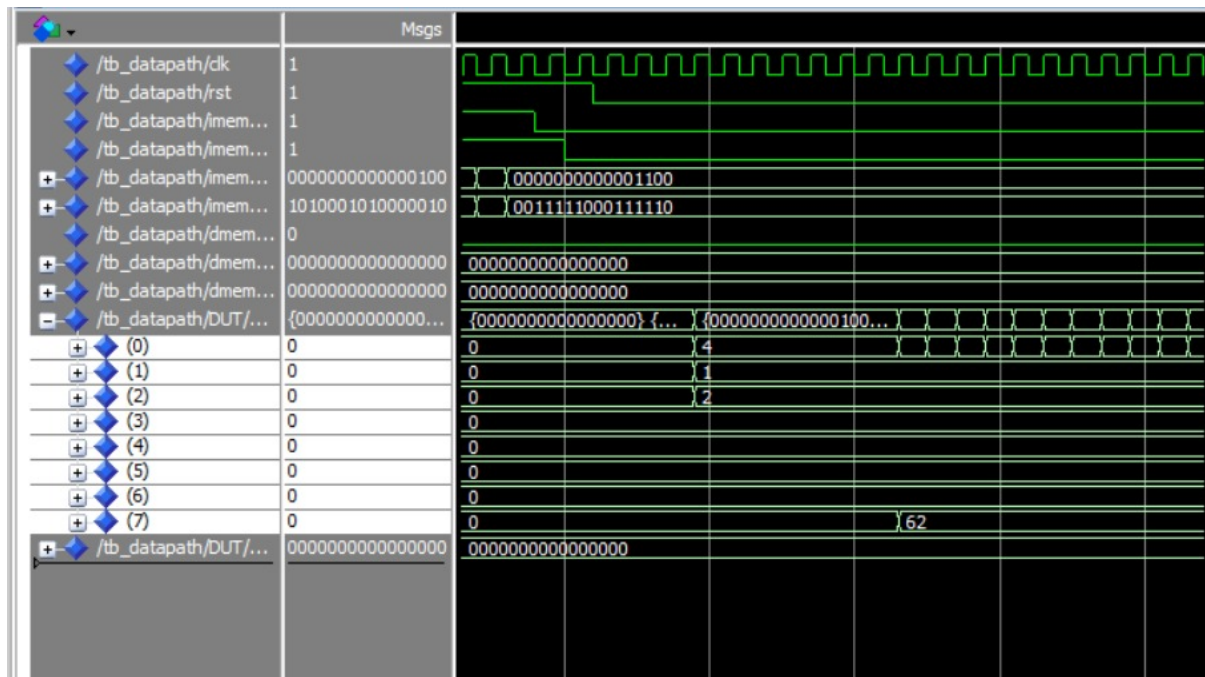


Figure 17: Simulation waveform for the BLE instruction (Branch if Less than or Equal).

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
10100010100000010 // ble r1, r2, 2
00110110000001100 // lli r3, 12
10100110110000010 // ble r3, r3, 2
00111000000000100 // lli r4, 4
00111110001111110 // lli r7, 62

```

3.18. Test 18: BLT

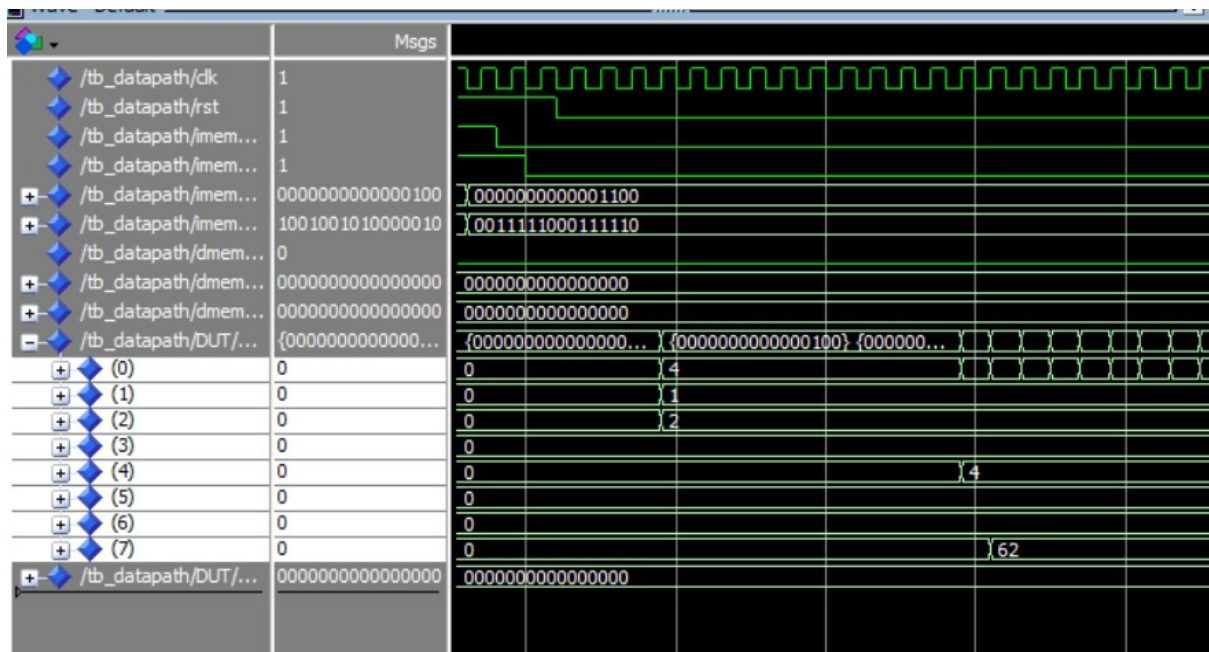


Figure 18: Simulation waveform for the BLT instruction (Branch if Less Than).

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
10010010100000010 // blt r1, r2, 2
00110110000001100 // lli r3, 12
10010110110000010 // blt r3, r3, 2
00111000000000100 // lli r4, 4
00111110001111110 // lli r7, 62

```

3.19. Test 19: LM

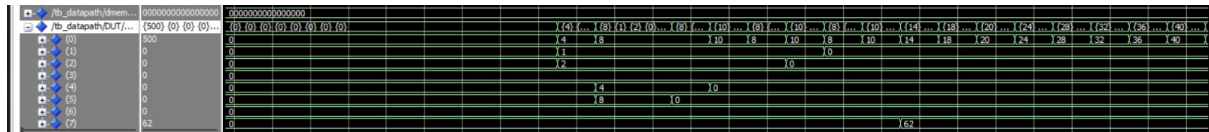


Figure 19: Simulation waveform for the LM (load multiple).

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
00111010000001000 // lli r5, 8
00111000000000100 // lli r4, 4
01100010011111100 // LM
00111110001111110 // lli r7, 62

```

3.20. Test 20: JLR

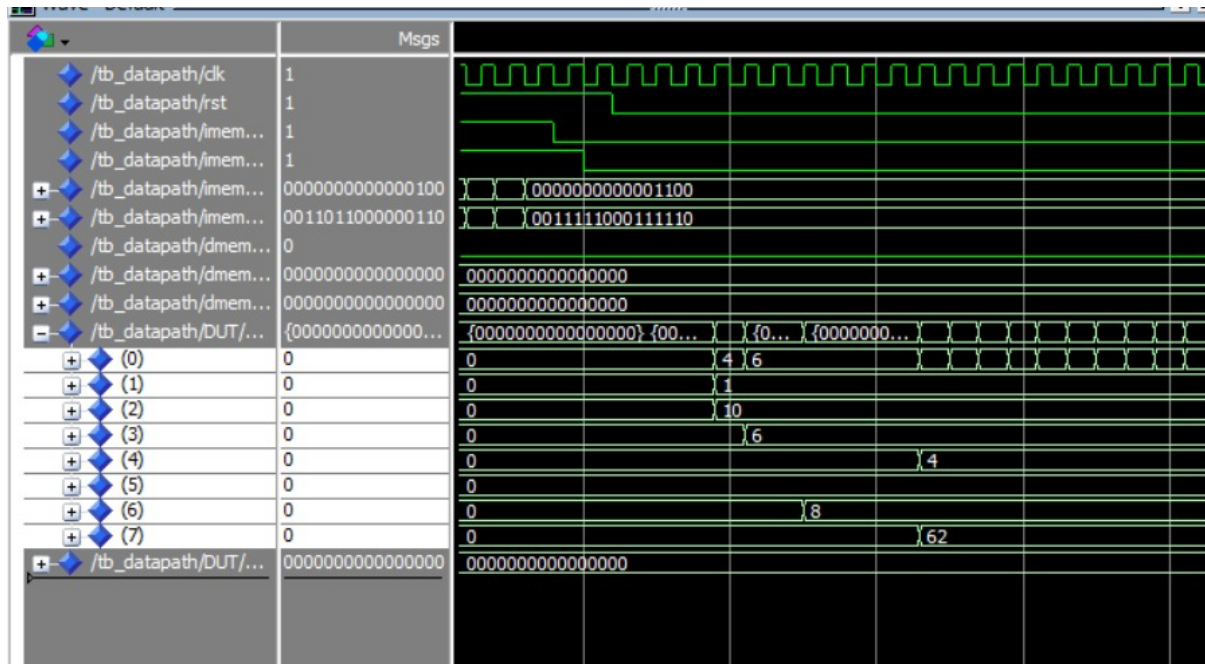


Figure 20: Simulation waveform for the JLR instruction (Jump and Link Register).

```

0011001000000001 // lli r1, 1
0011010000001010 // lli r2, 10
0011011000000110 // lli r3, 6
1101110010000000 // jlr r6, r2
0011011000001100 // lli r3, 12
0011100000000100 // lli r4, 4
0011111000111110 // lli r7, 62

```

3.21. Test 21: JRI

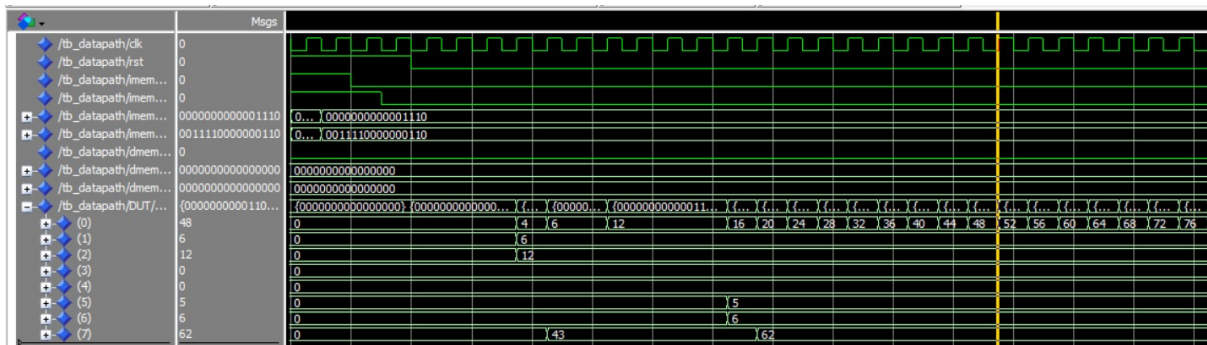


Figure 21: Simulation waveform for the JRI instruction (Jump to register to immediate).

```

00110010000000110 // lli r1, 6
0011010000001100 // lli r2, 12
0011111000101011 // lli r7, 43
1111010000000000 // jri r2, 0
0011111000111110 // lli r7, 62
0011100000000100 // lli r4, 4
0011101000000101 // lli r5, 5
0011110000000110 // lli r6, 6

```

3.22. Test 22: JAL

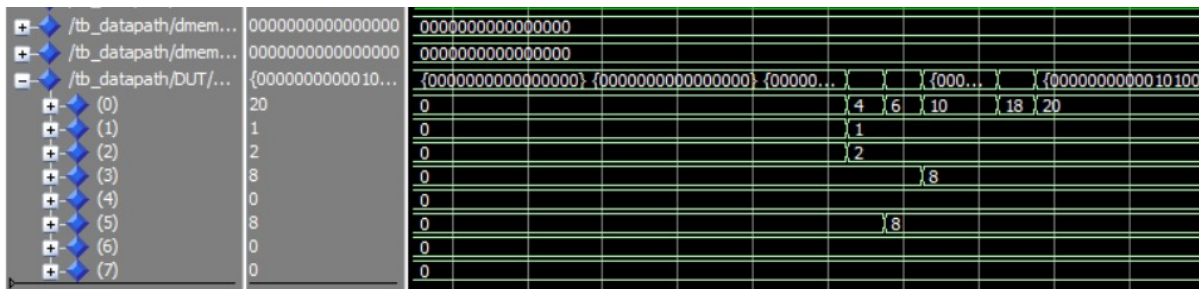


Figure 22: Simulation waveform for the JAL instruction (Jump and link).

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
00111010000001000 // lli r5, 8
11000110000000010 // jal r3, 2
00110110000001100 // lli r3, 12
00111000000000100 // lli r4, 4
00111110001111110 // lli r7, 62

```

3.23. Test 23: R0

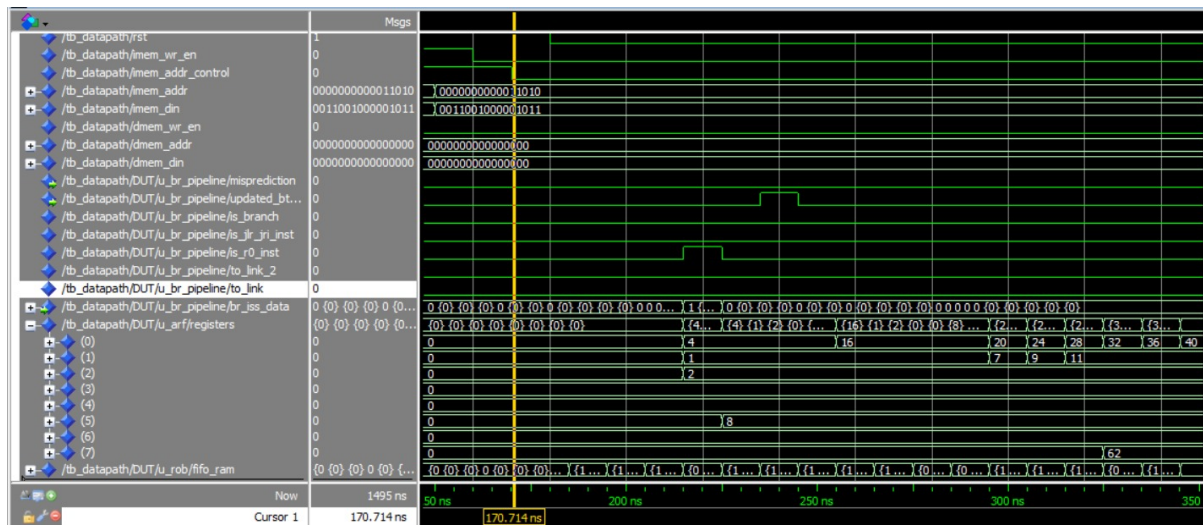


Figure 23: Simulation waveform for the R0.

```

00110010000000001 // lli r1, 1
00110100000000010 // lli r2, 2
00111010000001000 // lli r5, 8
00001010000001000 // adi r0, r5, 8
00110010000000010 // lli r1, 2
00110010000000011 // lli r1, 3
00110010000000100 // lli r1, 4
00110010000000101 // lli r1, 5
00110010000000110 // lli r1, 6
00110010000000111 // lli r1, 7
00110010000001000 // lli r1, 8
00110010000001001 // lli r1, 9
00110010000001010 // lli r1, 10
00110010000001011 // lli r1, 11

```

4 Conclusion

The implemented processor backend demonstrated that a compact IITB-RISC superscalar core can be realized using distributed reservation stations, a unified common data bus, and in-order retirement through a reorder buffer. The design successfully separated operand readiness from architectural commitment, allowing instructions to execute out of order while preserving precise program order at commit.

The final architecture integrated dynamic scheduling, split flag renaming, store buffering, and store-to-load forwarding into a cohesive VHDL-based microarchitecture. The resulting system provided a realistic and extensible foundation for further work on branch recovery, memory dependence prediction, and additional performance optimizations.

5 Assembler

A custom python assembler was also made for rapid testing. The following is the code for the assembler.

```

1 import sys
2 import argparse
3
4 def reg_to_bin(reg_str):
5     """Converts register string (e.g., 'r0', 'R7') to 3-bit binary."""
6     reg_str = reg_str.lower().strip()
7     if not reg_str.startswith('r'):
8         raise ValueError(f"Invalid register format: {reg_str}")
9     reg_num = int(reg_str[1:])
10    if reg_num < 0 or reg_num > 7:
11        raise ValueError(f"Register out of bounds: {reg_str}")
12    return format(reg_num, '03b')
13
14 def imm_to_bin(imm_str, bits):
15
16     imm_val = int(imm_str.strip())
17
18     min_val = -(1 << (bits - 1))
19     max_val = (1 << bits) - 1 # Allow unsigned max for masks like LM/SM
20
21     if imm_val < min_val or imm_val > max_val:
22         raise ValueError(f"Immediate {imm_val} out of bounds for {bits}
23         }-bit value.")
24
25     # Apply two's complement mask
26     bin_str = format(imm_val & ((1 << bits) - 1), f'0{bits}b')
27     return bin_str
28
29 def assemble_instruction(line):
30
31     line = line.split('#')[0].strip() # Ignore comments
32     if not line: return None
33
34     parts = line.replace(',', ' ').split()
35     op = parts[0].lower()
36
37     r_type_ops = ['ada', 'adc', 'adz', 'awc', 'aca', 'acc', 'acz', 'acw',
38     ,
39     'ndu', 'ndc', 'ndz', 'ncu', 'ncc', 'ncz']
40
41     if op in r_type_ops:

```

```

41     rc = reg_to_bin(parts[1])
42     ra = reg_to_bin(parts[2])
43     rb = reg_to_bin(parts[3])
44
45
46     opcode = '0001' if op.startswith('a') else '0010'
47
48
49     comp = '1' if op[1] == 'c' else '0'
50
51
52
53     if op in ['awc', 'acw']:
54         cz = '11'
55     elif op.endswith('c'):
56         cz = '10'
57     elif op.endswith('z'):
58         cz = '01'
59     else:
60         cz = '00'
61
62     return f"{opcode}{ra}{rb}{rc}{comp}{cz}"
63
64
65 elif op == 'adi':
66
67     rb = reg_to_bin(parts[1])
68     ra = reg_to_bin(parts[2])
69     imm6 = imm_to_bin(parts[3], 6)
70     return f"0000{ra}{rb}{imm6}"
71
72 elif op in ['lw', 'sw', 'beq', 'blt', 'ble']:
73
74     ra = reg_to_bin(parts[1])
75     rb = reg_to_bin(parts[2])
76     imm6 = imm_to_bin(parts[3], 6)
77
78     if op == 'lw': opcode = '0100'
79     elif op == 'sw': opcode = '0101'
80     elif op == 'beq': opcode = '1000'
81     elif op == 'blt': opcode = '1001'
82     elif op == 'ble': opcode = '1010'
83
84     return f"{opcode}{ra}{rb}{imm6}"
85
86 elif op == 'jlr':
87
88     ra = reg_to_bin(parts[1])

```

```

89     rb = reg_to_bin(parts[2])
90     return f"1101{ra}{rb}000000"
91
92
93     elif op in ['lli', 'jri', 'jal', 'jalr']:
94
95         ra = reg_to_bin(parts[1])
96         imm9 = imm_to_bin(parts[2], 9)
97
98         if op == 'lli': opcode = '0011'
99         elif op in ['jal', 'jalr']: opcode = '1100'
100        elif op == 'jri': opcode = '1111'
101
102        return f"{opcode}{ra}{imm9}"
103
104        elif op in ['lm', 'lmf', 'sm', 'smf']:
105
106        ra = reg_to_bin(parts[1])
107        imm8 = imm_to_bin(parts[2], 8)
108
109        opcode = '0110' if op.startswith('l') else '0111'
110        flag = '1' if op.endswith('f') else '0'
111
112        return f"{opcode}{ra}{flag}{imm8}"
113
114        else:
115            raise ValueError(f"Unknown instruction: {op}")
116
117    def assemble_code(assembly_text):
118
119        machine_code = []
120        for line_num, line in enumerate(assembly_text.strip().split('\n'),
121        1):
122            original_line = line.strip()
123            if not original_line or original_line.startswith('#'):
124                continue # Skip empty lines and full-line comments
125
126            try:
127                bin_code = assemble_instruction(original_line)
128                if bin_code:
129                    machine_code.append((original_line, bin_code))
130            except Exception as e:
131                machine_code.append((original_line, f"ERROR on line {
132        line_num}: {str(e)}"))
133
134        return machine_code

```

```

135 if __name__ == "__main__":
136
137     parser = argparse.ArgumentParser(description="Assembler for IITB-
138     RISC-26 Architecture.")
139     parser.add_argument("input_file", nargs="?", default="t1.txt", help
140     ="Path to the input assembly file (default: input.txt)")
141     parser.add_argument("output_file", nargs="?", default="t2.txt",
142     help="Path to the output machine code file (default: output.txt)")
143
144     args = parser.parse_args()
145
146     # Read from input file
147     try:
148         with open(args.input_file, 'r') as infile:
149             assembly_source = infile.read()
150     except FileNotFoundError:
151         print(f"Error: Could not find input file '{args.input_file}'.")
152         sys.exit(1)
153
154     print(f"Reading assembly from '{args.input_file}'...")
155
156     # Assemble the code
157     results = assemble_code(assembly_source)
158     error_count = 0
159     success_count = 0
160
161     # Write to output file
162     try:
163         with open(args.output_file, 'w') as outfile:
164             for instruction, binary in results:
165                 if "ERROR" in binary:
166                     # outfile.write(f"// {binary}\n")
167                     print(binary) # Also print errors to console for
168                     immediate visibility
169                     error_count += 1
170                 else:
171
172                     outfile.write(f"{binary} // {instruction}\n")
173                     success_count += 1
174
175     print(f"Successfully assembled {success_count} instructions.")
176     if error_count > 0:
177         print(f"Completed with {error_count} errors. Check the
178         output file for details.")
179         print(f"Machine code written to '{args.output_file}'.")
180
181     except IOError as e:

```

```
178     print(f"Error writing to output file '{args.output_file}': {e}"  
    )  
179     sys.exit(1)
```

Listing 1: Assembler

6 Work Distribution

Table 1: Project work distribution

Team Member	Roll Number	Primary Responsibilities
Advay Bapat	23B1225	Branch Predictor, 3 Execution Pipelines, Fetch
Apoorv Goyal	23B1266	3 Reservation Stations, Store Buffer, 3 Schedulers
Shreya Nigam	23B1258	ROB, ARF, RRF, RRF for Flag, Commit logic
Suketu Patni	23B1299	Dispatch, Decoder, Decode Stage, Report

Extensive debugging and testing was performed by all the team members equally.